

Verborum — Frontend-Backend Integration

Target location of this file: backend repo `verborum_ms/docs/integration/frontend-backend-integration.md` (referenced from the Android and future KMP repos — single copy, no duplicates)

Related: *Verborum Android Development* (`verborum_android/docs/android-development.md`),
backend agent docs (`verborum_ms/docs/agent/`)

1. The Platform Picture

One backend, three client applications in **three separate repositories**:

Client	Technology	Repo	Status
Android	Native Kotlin, Jetpack Compose	<code>verborum_android</code>	Working (pre-auth)
WebApp	KMP , Compose Multiplatform	(to be created)	Planned — next client
iOS	KMP , Compose Multiplatform	(to be created)	Planned — after web

Android stays native permanently. The two KMP projects are independent of the Android codebase — **nothing is shared between clients except the contracts in this document**. That is precisely why this document exists: with three independent implementations, whatever is not specified here will be invented three times, differently.

Delivery policy: work streams run in parallel and must not block each other, but **nothing is released until Android + Backend are ready as a pair**. Web-required infrastructure (CORS, browser token handling) is built on the backend at its natural phase even if Android releases first — readiness on the BE side is decoupled from client release order.

2. Backend Topology — a Client's View

```
Client (Android / Web / iOS)
|  HTTPS + Authorization: Bearer <JWT>
▼
API Gateway (Spring Cloud Gateway, single public origin)    ← BE Phase 5
├─ /dictionaries/**, /words/**    → ms_dictionary (8085)
├─ /users/**                      → ms_user          (8086)    ← BE Phase 2
└─ /marketplace/**                → ms_marketplace (8087)    ← BE Phase 4
Keycloak (auth server, Google SSO federated)                ← BE Phase 3
RabbitMQ (verborum.events)  – INTERNAL ONLY, never client-visible
```

Rules for every client:

- Clients talk to **the gateway and Keycloak only**. Individual service ports exist only in local development.
- RabbitMQ is backend-internal. Clients see its *effects* (§9), never the broker.
- Google sign-in is federated **behind Keycloak** — no client ever integrates Google directly.

3. API Contract

REST + JSON. Two response shapes:

Mutations return the standard envelope:

```
{ "status": 201, "message": "Dictionary saved successfully: <id>", "path": "/dictionaries/", "timestamp": "..." }
```

Errors (all services, all cases — validation, not-found, malformed body):

```
{ "status": 404, "error": "RecordNotFoundException", "errorDetail": "Dictionary not found for ID: ...", "path": "/words", "timestamp": "..." }
```

Reads return the DTO or DTO list directly, no envelope. Semantics every client must code against: 400 validation/malformed body · 401 missing/invalid token (post-Phase-3) · 404 unknown record · empty list (not 404) for batch/list endpoints with no matches.

3.1 Identity of records

All IDs are **client-generated UUID strings**. The backend never generates IDs. This is what makes offline creation possible and is non-negotiable for every client. After auth (§6), the *owner* of a record is always the JWT subject — any client-supplied `userId` is ignored by the backend (BE task P3-05).

3.2 Endpoint catalog (client-relevant)

ms_dictionary — live today

Method & path	Purpose
POST /dictionaries/ · PUT /dictionaries/	Create / update a dictionary
DELETE /dictionaries/{dictionaryId}	Delete (words cascade server-side)
GET /dictionaries/{userId}	All dictionaries of a user
GET /dictionaries/dictionary/{dictionaryId}	Single fetch
GET /dictionaries/batch?ids=a,b,c	Batch fetch
POST /words · PUT /words	Save words as bundles: [{"dictionaryId": "...", "words": [...]}]
DELETE /words/{wordId} · DELETE /words/dictionary/{dictionaryId}	Deletes
GET /words/dictionary/{id} · GET /words/user/{userId} · GET /words/batch?ids=...	Reads

ms_user — BE Phase 2: POST/PUT /users/ , GET/DELETE /users/{userId} , vault: GET/POST /users/{userId}/vault , DELETE /users/{userId}/vault/{dictionaryId} .

ms_marketplace — BE Phase 4: GET /marketplace/dictionaries (paginated), .../popular , .../language?from=EN&to=DE , POST /marketplace/dictionaries/{id}/import .

ms_autofil — BE Phase 6 (V2): GET /autofil?word=...&from=...&to=... → ranked community translations.

4. Data Contracts

4.1 Supported languages — one list, three clients, one backend

The backend property `supported.languages` is the **single source of truth**; every client's language enum must be a subset of it. Target list (19, after the Android expansion):

```
EN, DE, FR, ES, IT, PT, NL, TR, AZ, LT, PL, UK, AR, FA, JA, ZH, KO, EL, RU
```

Chinese is **Simplified only** (`zh`). Update procedure: backend property first (accepting a superset is harmless), then clients. *Known live gap*: the backend still validates the original 8 — pt/nl uploads fail today; the property update is required backend work.

4.2 Word content — the canonical meta schema

Defined by the Android implementation and documented normatively in the Android doc §4. Summary of what web/iOS must reproduce exactly:

- `word` / `translation` columns: **JSON array of per-meaning surfaces** — `["kaufen","erwerben"]`.
- `word_meta` / `translation_meta`: one JSON object `{"lang","type?","genders?","fields?"}` with all lists **index-aligned to the surfaces array**; keys empty everywhere are omitted; **unknown keys are ignored** (the evolution rule).
- Field keys currently: `reading, plural, feminine, comparative, superlative, present, past, past3, participle, aux, aspect, root, stem, measure, class, polite` (the last seven arrive with the 19-language expansion).

Backend action item: `Word.java` and `verborum.md` currently document a different meta shape (`partOfSpeech` / `example` / `notes`) written before the client schema was known. The backend stores the column opaquely so nothing breaks at runtime, but the docs must adopt the schema above before any other consumer (web, iOS, ms_autofil) implements against the fiction.

5. Sync & Offline Model

Sync internals are **client-internal** — the backend imposes no sync protocol — but any offline-capable client must respect the shared contract: client-generated UUIDs; deletes are real `DELETE` calls (remote-first, local hard-delete after confirmation); updates are full upserts via `PUT`.

- **Android** is offline-first (Room + `isSynced` flags + tombstones, upload-then-download, local-unsynced-wins). This design is documented in the Android doc §5 and **recommended** as the reference for iOS if it goes offline-capable.
- **Web** is expected online-only (no durable local store; browser storage is not trusted).
- **At release Android is online + login required.** Offline-first remains as the resilience layer under a logged-in session — it is not an anonymous mode.

6. Authentication Contract (normative for all three clients)

Keycloak realm `verborum`; Google federated behind it. **Authorization Code + PKCE for every platform.** The first client to implement auth implements *this spec* — not a precedent of its own making.

6.1 Flow & clients

	Android	iOS	Web
Keycloak client	<code>verborum-app</code> (public)	<code>verborum-app</code> (public)	<code>verborum-web</code> (public) or BFF confidential — §6.3
Login UI	AppAuth / Custom Tabs	ASWebAuthenticationSession	Browser redirect
Token storage	EncryptedSharedPreferences / DataStore + Keystore	Keychain	§6.3
Refresh scope	<code>offline_access</code>	<code>offline_access</code>	none (short sessions)

6.2 Token policy (configured per Keycloak client — design them together, BE P3-02)

Mobile: short access tokens (≈5 min), long offline-capable refresh tokens — a device offline for days must resume sync without re-login. Web: short everything; re-authentication is acceptable in a browser. All clients: send `Authorization: Bearer <access>`; refresh on 401 once, then surface login.

6.3 Web token storage — decision pending, backend must stay compatible with both

Browsers have no secure token storage. Two options:

1. **BFF (recommended)**: a thin server component of the web app holds tokens; the browser gets an `httpOnly` session cookie. Most secure; means the KMP web repo ships with a small server — this shapes that repo's structure and should be decided before it is created.
2. **Pure SPA**: tokens in memory only (never `localStorage`), silent re-auth on reload.

The gateway must therefore accept standard `Authorization: Bearer` (mobile, BFF-to-gateway) regardless of which option the web app picks — which it does by design.

6.4 Guest-data migration (decide once — implement per client with local data)

A client that allowed local data creation before login (Android today) migrates on first login **client-side**: rewrite the local owner id from the guest UUID (`00000000-0000-0000-0000-000000000000`) to the JWT subject, mark all rows unsynced, run the normal upload. No special backend endpoint exists or is needed — client-generated UUIDs make the records globally valid already. The guest UUID must never reach the server post-auth.

7. Web-Specific Backend Readiness (build at the right time, block nobody)

Two things only browsers need — both land with their natural backend phases regardless of whether Android releases first:

- **CORS** (BE Phase 5, gateway): allowed origins per environment for the web app's domains; `Authorization` + `Content-Type` headers; all verbs. Keycloak realm clients get matching *Web Origins*. Mobile clients are unaffected.
- **Cookie/session tolerance**: only relevant if BFF is chosen — and then it is the BFF's concern, not the gateway's; the gateway keeps seeing bearer tokens (§6.3).

8. Environments

Environment	API origin	Keycloak	Transport
dev	<code>http://localhost:8085-8087</code> or LAN (per-service, no gateway)	<code>http://localhost:8180</code>	cleartext allowed
staging	gateway origin, HTTPS	staging realm	HTTPS only
prod	gateway origin, HTTPS	prod realm	HTTPS only

Per-platform mechanics: Android build types / product flavors (`BuildConfig`); KMP projects use build-time configuration per target. The current Android hardcoded LAN URL (`192.168.0.241:8085` , cleartext in release) is a dev artifact scheduled for removal (Android roadmap A3).

9. Backend Events as Client-Visible Behavior

Clients never see RabbitMQ, but they rely on the behavior it produces:

Client action	Backend event chain	Visible effect
Set dictionary <code>isPublic = true/false</code>	<code>dictionary.visibility.*</code> → <code>ms_marketplace</code>	Listing appears/disappears in marketplace
Delete account	<code>user.deleted</code> → <code>ms_dictionary</code> , <code>ms_marketplace</code>	All server data of the user gone
Import from marketplace	<code>dictionary.imported</code> → <code>ms_user vault</code>	Dictionary arrives via normal sync
Create words	<code>word.created</code> → <code>ms_autofil (V2)</code>	Community suggestion counts grow

Consequence for clients: these effects are **eventually consistent** — an imported dictionary or a new listing may take a moment. Do not design UI that assumes synchronous propagation.

10. Compatibility Rules

1. **Additive-only** API evolution: new fields, new endpoints; never repurpose or remove while any client version is live.
2. Meta schema evolves by **adding keys**; parsers ignore unknown keys (already true on Android; mandatory for web/iOS).
3. Language list grows backend-first (§4.1).
4. Envelope shapes (§3) are frozen; new information means new fields, not changed ones.

11. Milestone Mapping

Backend milestone	Unblocks
Language property → 19 codes (small task, do early)	Android A1 release; fixes live pt/nl gap
Phase 2 (<code>ms_user</code>) + Phase 3 (Keycloak/JWT)	Android A2 login; any client auth
Phase 4 (<code>ms_marketplace</code>)	Android A4 forum tab; web marketplace pages
Phase 5 (gateway + CORS)	Android A3 final networking; web development start
Phase 6 (<code>ms_autofil</code> , V2)	Suggestion features on all clients

Recommended additions to the backend roadmap: the language-list task, the meta-doc correction (§4.2), and an auth-contract section in `security.md` mirroring §6 of this document.